Task 5: Preprocessor, Macros and Conditional Compilation

Student name	Sk.Thohith	Register number	1912260555
GitHub repository	https://github.com/[username]/CSA02-[RegisterNumber]-Assignment1/tree/main/Task05	Compile command	gcc -Wall -Wextra -std=c11 -O2 main.c -o task
Compiler	Online C compiler (screenshot); own checks GCC 13.3.0	Date	26 October 2026

5.1 Problem analysis

The program computes an electricity bill in paise from a number of units using a three-slab tariff plus a fixed charge, and prints it for five test values (0, 100, 150, 200, 350). The tariff constants are not written inside the logic: they are preprocessor macros, wrapped in a header guard, and two debug messages are compiled in only when `DEBUG` is defined. The task therefore exercises **#include, #define, #ifndef/#endif (include guard) and #ifdef (conditional compilation)**, together with a function prototype and a `long` accumulator for money in paise.

Tariff encoded in the macros: units 1-100 at RATE1 = 0 paise; units 101-200 at RATE2 = 225 paise (Rs 2.25); units above 200 at RATE3 = 450 paise (Rs 4.50); plus FIXED_CHARGE_PAISE = 5000 (Rs 50.00). The first slab is free; RATE1_PAISE is defined but never used in the visible code, because a zero rate adds nothing.

5.2 Data types, constants and preprocessor concepts

Directive / item	Form	Purpose
#include <stdio.h>	Standard header	Declares printf
#ifndef BILLING_H / #define BILLING_H / #endif	Include guard around the macros and the prototype	Prevents duplicate definitions if the header part is included twice (needed once the code is split into billing.h and main.c)
SLAB1_UNITS, SLAB2_UNITS	#define 100, 200	Slab boundaries in one place
RATE1/2/3_PAISE	#define 0, 225, 450	Rates in paise per unit (integers: exact money)
FIXED_CHARGE_PAISE	#define 5000L	The L suffix makes the constant a long, matching the long accumulator
#ifdef DEBUG ... #endif	Conditional compilation (two places)	Debug printf lines exist in the program only when compiled with -DDEBUG
long compute_bill(int units)	Prototype + definition	Returns paise as long; units is int
(long)(units - SLAB2_UNITS) * RATE3_PAISE	Cast before multiply	Forces the product to be computed in long, not int
bill / 100, bill % 100 with %ld.%02ld	Integer division and remainder	Rupees and two-digit paise without floating point

5.3 Source code (main.c, lines 1-68)

```
#include <stdio.h>

#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units);

#endif

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

#ifdef DEBUG
    printf("[DEBUG] Units received: %d\n", units);
#endif

    if (units > SLAB2_UNITS)
    {
        paise += (long)(units - SLAB2_UNITS) * RATE3_...;
    }

    if (units > SLAB1_UNITS)
    {
        int slab2;

        if (units < SLAB2_UNITS)
            slab2 = units - SLAB1_UNITS;
        else
            slab2 = SLAB2_UNITS - SLAB1_UNITS;

        paise += (long)slab2 * RATE2_PAISE;
    }

#ifdef DEBUG
    printf("[DEBUG] Total bill in paise: %ld\n", pai...);
#endif

    return paise;
}

int main(void)
{
    int units[] = {0, 100, 150, 200, 350};
    int n = sizeof(units) / sizeof(units[0]);

    printf("ELECTRICITY BILL CALCULATION\n");
    printf("----------------------------\n");

    for (int i = 0; i < n; i++)
    {
        long bill = compute_bill(units[i]);

        printf("%3d units -> Rs %ld.%02ld\n",
               units[i], bill / 100, bill % 100);
    }

    return 0;
}
```

Two lines are clipped at the right edge of the screenshot (line 29 and line 45, marked ...). Judging from the surrounding code, line 29 ends with RATE3_PAISE; and line 45 ends with paise); the program output confirms this. Copy the exact lines from your main.c.

5.4 What the preprocessor does

The preprocessor runs before the compiler and substitutes text. I ran `gcc -E -P` on the code (without DEBUG). The macros disappear and the debug lines are removed entirely:

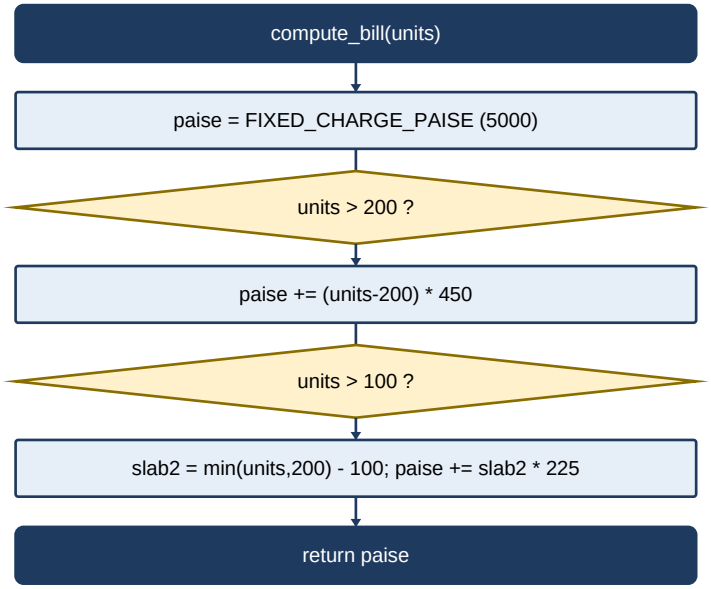
```
long compute_bill(int units);
long compute_bill(int units)
{
    long paise = 5000L;
    if (units > 200)
        paise += (long)(units - 200) * 450;
    if (units > 100)
        ...  paise += (long)slab2 * 225;
    return paise;
}
```

Because the substitution is purely textual, a macro has no type and no scope, and a macro that expands to an expression should be parenthesised, for example `#define LIMIT (SLAB2_UNITS + 1)`. Every macro here is a single integer literal, so the expansion is safe.

5.5 Pseudocode

```
FUNCTION compute_bill(units):
    paise <- FIXED_CHARGE                              // 5000
    IF DEBUG THEN PRINT units
    IF units > 200 THEN paise += (units - 200) * 450
    IF units > 100 THEN
        slab2 <- (units < 200) ? units - 100 : 200 - 100
        paise += slab2 * 225
    IF DEBUG THEN PRINT paise
    RETURN paise
MAIN: FOR each u in {0,100,150,200,350}: bill <- compute_bill(u)
      PRINT u, "Rs", bill DIV 100, ".", bill MOD 100 (2 digits)
```

UML activity diagram - Task 5 (compute_bill)



5.6 Output evidence

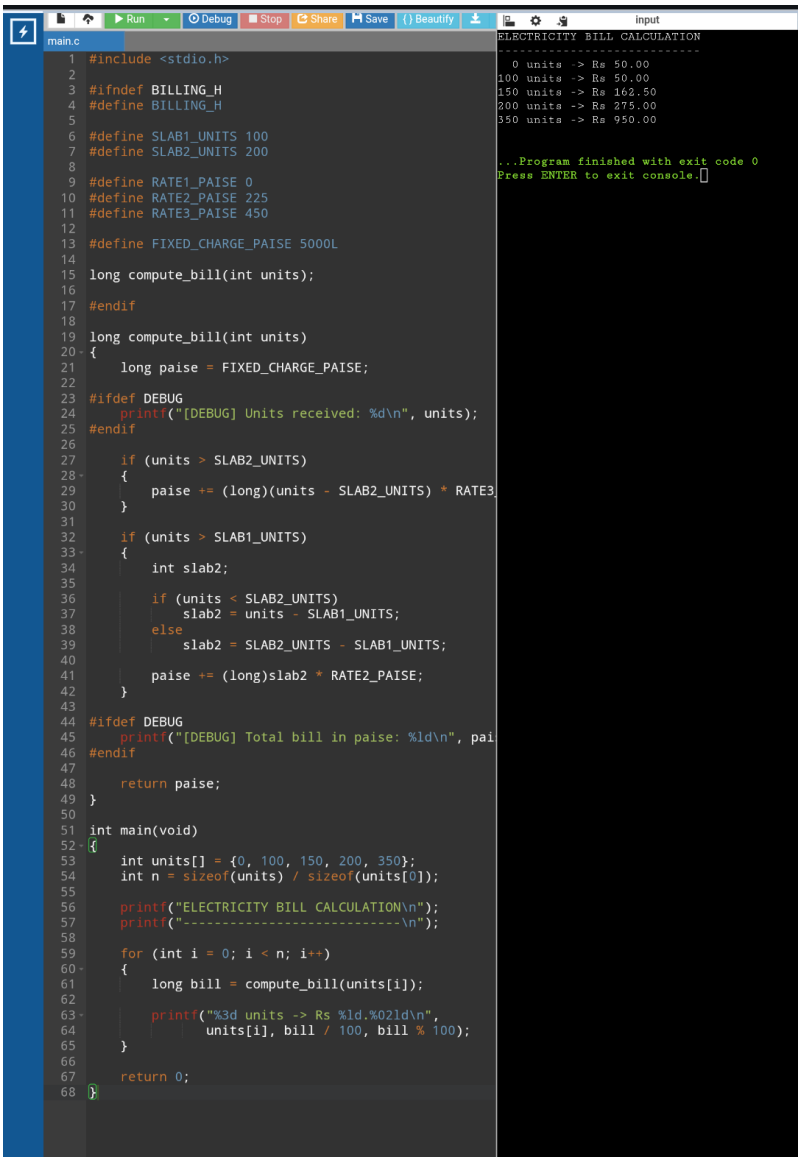


Figure 1 - Task 5 source (left) and console output (right).

5.7 Test table (expected vs actual)

#	Units	Expected (hand calculation, paise)	Actual (screenshot)	Result
1	0	Fixed charge only: 5000 = Rs 50.00	Rs 50.00	Pass
2	100 (boundary, not above slab 1)	5000 = Rs 50.00 (first 100 units free)	Rs 50.00	Pass
3	150	5000 + 50 x 225 = 16250 = Rs 162.50	Rs 162.50	Pass
4	200 (boundary of slab 2)	5000 + 100 x 225 = 27500 = Rs 275.00	Rs 275.00	Pass
5	350	5000 + 100 x 225 + 150 x 450 = 5000 + 22500 + 67500 = 95000 = Rs 950.00	Rs 950.00	Pass

Additional checks (my own run of the reconstructed program, GCC 13.3.0; repeat on your machine).

Case	Expected	Actual (my run)
101 (just above slab 1)	5000 + 1 x 225 = 5225 = Rs 52.25	5225
201 (just above slab 2)	5000 + 22500 + 450 = 27950 = Rs 279.50	27950
313 units	5000 + 22500 + 113 x 450 = 78350 = Rs 783.50	78350

Case	Expected	Actual (my run)
1000 units	5000 + 22500 + 800 x 450 = 387500 = Rs 3875.00	387500
Compiled with -DDEBUG	Extra lines "[DEBUG] Units received: N" and "[DEBUG] Total bill in paise: P" before each result	Printed for every unit value
Negative units, e.g. -5	No validation in this function: neither slab test is true, bill = 5000	Rs 50.00 (limitation)

Link to Task 11. 313 units gives Rs 783.50, the same figure as the Task 11 screenshot (energy Rs 733.50 + fixed Rs 50.00), and 150 units gives Rs 162.50, the per-household bill used in Task 10. This indicates that Task 11 uses the same tariff; confirm against your Task 11 source.

Effect of DEBUG on the program. Object code size at -O2: 610 bytes without DEBUG and 689 bytes with -DDEBUG (my measurement), so the debug code is genuinely absent from the normal build rather than skipped at run time. DEBUG was not defined in the screenshot run, so no debug lines appear in the console output.

Limitations. (1) No input validation for negative units. (2) `long` is 8 bytes on 64-bit Linux but 4 bytes on 64-bit Windows compilers; with 4-byte long, very large unit counts times 450 could overflow, so use `long long` if portability matters. (3) The guard macro protects only against double inclusion; here everything is in one file, so it demonstrates the technique rather than being strictly needed.

5.8 Alternative concept: typed constants (const / enum) and a runtime debug flag instead of macros

Instead of `#define`, the constants can be typed objects, and instead of `#ifdef DEBUG` a runtime flag can be tested:

```
enum { SLAB1_UNITS = 100, SLAB2_UNITS = 200 };      /* integer constants, scoped, visible to a debugger */
static const long RATE2_PAISE = 225, RATE3_PAISE = 450, FIXED_CHARGE_PAISE = 5000L;
static int debug = 0;                                 /* set from argv or an environment variable */
...
if (debug) printf("[DEBUG] Units received: %d\n", units);
```

	#define macros + #ifdef (implemented)	const / enum + runtime flag
Type checking	None (text substitution)	Full: compiler checks types and scope
Debugger visibility	Name is gone after preprocessing	Name and value visible
Debug code in release build	Removed entirely (610 vs 689 bytes)	Stays in the binary; costs one branch per call
Switch without recompiling	No (needs -DDEBUG)	Yes
Time complexity of compute_bill	O(1)	O(1) (plus one test of the flag)
Space complexity	O(1); constants are inlined in the code	O(1); a few constants in read-only data and one flag variable
Use in array sizes / case labels	Yes	enum yes; const int not in standard C

Measurement (my benchmark: 2 x 10^8 calls of the tariff function with units cycling 0..511, GCC 13.3.0, single run on a cloud sandbox): macro version 0.648 s and const version 0.609 s at -O0; 0.438 s and 0.338 s at -O2. Both returned identical totals. The differences are small and come from the benchmark loop, not from the constants; with optimisation the compiler folds a `static const` into the code exactly as it does a macro. Choose between them for safety and debuggability, not for speed. The billing logic is O(1) per customer and O(n) for n customers in either style.

5.9 Design conclusions

Decision	Justification
Keep all tariff numbers in named constants	A tariff change edits one line; no magic numbers in the logic (SLAB, RATE, FIXED_CHARGE)
Money in integer paise, long accumulator, cast before multiply	Exact, and the (long) cast prevents int overflow of the product
Prefer const/enum for new code; keep macros for conditional compilation and include guards	Type safety and debugging for values; #ifdef is the right tool for code that must vanish from release builds
Guard debug output with DEBUG	Verified: 79 bytes of debug code removed from the normal build
Add input validation (units >= 0)	Closes the negative-units gap found in testing